Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering

Topic

Building Audio Model for Gender Detection

Course: Integrated Project (AI) - CO3101

Instructor: Ms. Võ Thanh Hùng

14th December 2025

Contents

1 Introduction

Artificial intelligence (AI) has a massive impact on several fields in the world. However, Nature Language Processing and Computer Vision are two major fields that everyone in the industry is paying attention to. Nevertheless, in the last few years, audio and speech have become more interested than ever. NVIDIA is currently finding intern for speech data recognition [1] and it is a signal to explore audio field more and more. There are 5 main auditory classes in audio signal classification which are noise, natural sound, artificial sound, speech and music, each classes has there own features that models can train and make prediction or do recognition [2].

Gender classification is a quite simple problem but still remains it's usability because it has plenty of applications that base from it. For instance, smart house system separates house utilities based on role like guest, home owner and it needs voice recognition to know which role is matched with the voice. This can avoid stranger from accessing some private functions that can cause harm to home owner's house and family. Gender classification is the first step to understand how AI model can learn and make prediction from just short audio records and from that, develop to word detection from voice and reply back. At the moment, this project works on the early stage of that implementation.

2 Problem

For gender classification, there are several models that can easily recognize gender from a second of the audio clip and also has API to access from open source library like HuggingFace [3]. The problem is it takes a huge amount of parameters just to classify gender from audio which is inefficient and impractical for integrating into a household system. Most of the public models are using transformer architecture which is powerful but redundant, so a lighter model contains only a convolutional neural network (CNN) as feature extractor and an attention block for context capturing are efficient and also lighter than full transformer block like Audio Transformer [4]. And that is the place for Light and Efficient Audio Network (LEAN) model. Firstly, this model uses plenty of CNN layers to extract feature (in paper [5], they used pretrained Yamnet and 2 layers of bidirectional LSTM to extract features. This is the more lighter and efficient more than full transformer block but it might not be effective for larger scale problem such as speech-to-text or earning call transcript analysis due to it's long context and all it has in the model is a small attention block to concatenate all the features it has extracted from the two feature extractor blocks. Nevertheless, for simple and small scale problem like gender recognition and audio tagging, it is necessary to reduce the parameters and trim the model to the lightest and the most efficient as possible.

3 Dataset Processing

The dataset contains 2 folders of audio records (.wav files) that have various duration. The male audio clips are hold in the directory /kaggle/input/gender-recognition-by-voiceoriginal/data/male and the female audio clips are in /kaggle/input/gender-recognition-by-voiceoriginal/data/female. First, the data is loaded in the df pandas DataFrame with *df["path"]* is the path of the file, for example, /kaggle/input/gender-recognition-by-voiceoriginal/data/female/arctic_a0001.wav, and *df["label"]* is the label for that path ("male" or "female").

3.1 Dataset Exploration

The dataset has 2 classes that are male and female. The former contains 10,4 thousand audio files while the later has 5768 thousands so basically, it is an imbalance dataset. For LEAN model, it takes 2 input types which are raw waveform and logmel spectrogram. I have process the data into these 2 and visualize it out as both types.

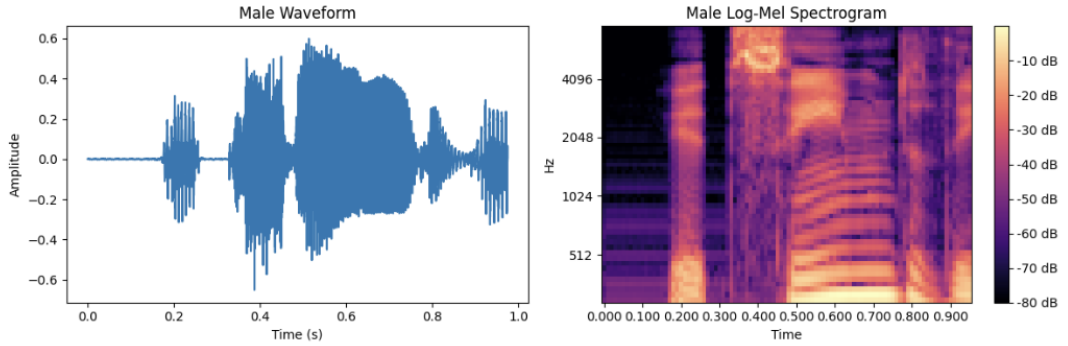

Figure 1: Waveform and Logmel Spectrogram as Time Sequence (Male)

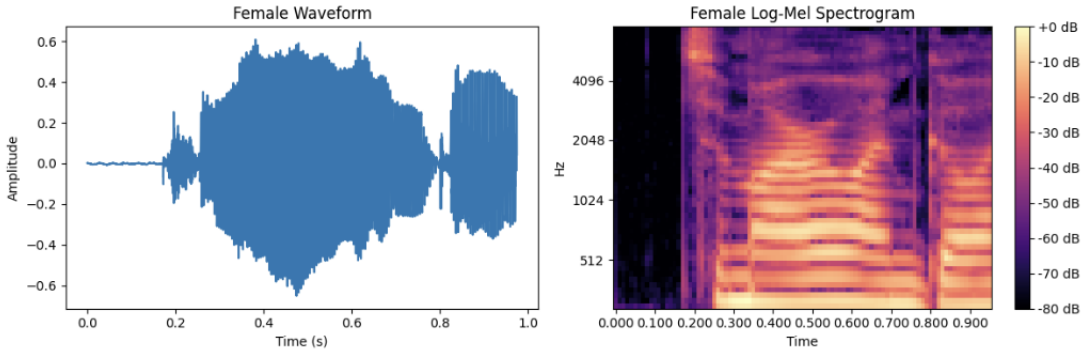

Figure 2: Waveform and Logmel Spectrogram as Time Sequence (Female)

In the paper [5], FSD50K is the dataset used to train but I think it quite confused because LEAN is a model that best for binary classification and FSD50K is a multi label dataset that contains multiple labels to recognize. It might be the reason why the metrics so low (below 0.5) which does not evaluate the model. In this project, I try to train this model with binary classes dataset to see if any change.

3.2 Processing

First, the dataset is severely imbalance when male audio file double the audio file of female so it might cause bias in training. In this project, I will balance it by undersampling the male samples equal to the number of female files which is 5768 samples so in total, we have 11536 samples.

Second, because the LEAN model takes 2 types of inputs is raw waveform and logmel spectrogram so I use 2 libraries to handle this transformation are Librosa and Pytorch. To transform sound to time sequence waveform and logmel spectrogram, I use these parameters:

- *N_MELS:* 64

  Explain: The height of spectrogram which is determine by the Mel filter bank. N_MELS = 64 compresses the frequency into 64 bands.

- *HOP_LENGTH:* 160

  Explain: The window slides forward by 160 samples (10ms) for each new frame.

- *WINDOW_LENGTH:* 400

  Explain: The function looks at 400 samples (25ms) at a time to analyze frequencies.

- *FRAMES:* 96

  Explain: The time that x-axis of the spectrogram. FRAMES $= 1+\frac{num\_samples - window\_length}{hop\_length}$

- *SR:* 16000 Hz

  Explain: Sample rate, frequency of the spectrogram that I want to transform into

- *SEGMENT_SAMPLES:* 15600

  Explain: The amount of samples that contained in the dataset. With SR, physical duration of an audio clip is $\frac{SEGMENT\_SAMPLES}{SR} = 0.975(ms)$

- *FMIN* and *FMAX:* [125,7500]

  Explain: the range of frequency that I will focus on. Outside that range will be ignored.

In additionally, I add noise and augmentation for training split because in the dataset, there are several duplicated samples (Maybe different lines but same vocal so the model may remember the features of that vocal and causing overfitting).

I have splited the dataset into 70/15/15 for train/test/val dataset for model training.

4 Model Implementation

4.1 Architecture

The model has 2 parts work as 2 individuals to extract as much features from the input as possible.

On the right side, it is a Yamnet CNN layers to extract features from logmel spectrogram. In paper [5], they have used VGG and PANN models in this layer with better performance but Yamnet is lighter and more understandable so I still use this as the feature extractor. After it goes through 256 units of projection layers, it will be embedded to 1024 vectors.

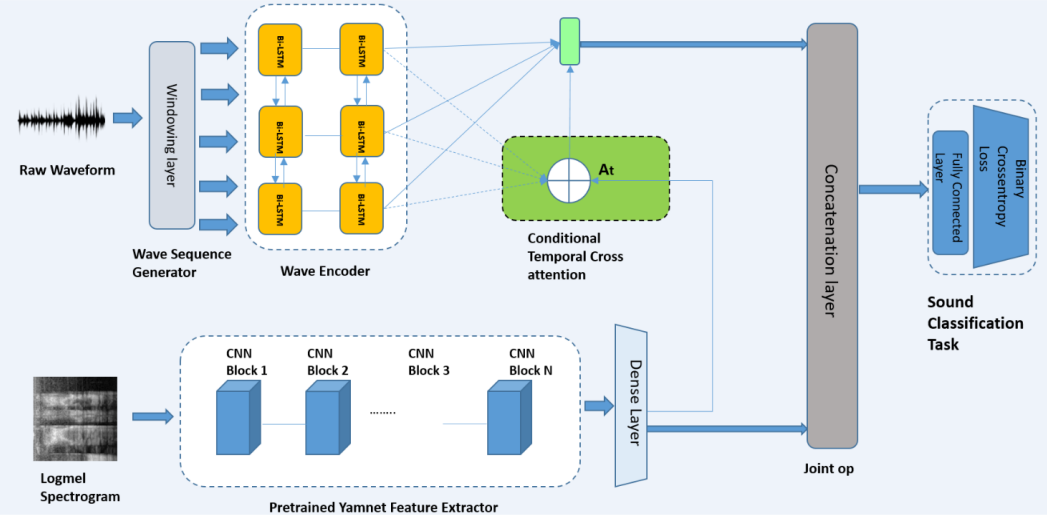

Fig. 1. Our Proposed Model Architecture (LEAN)

Figure 3: LEAN Architecture in [5]

On the left side, the model takes waveform signal and then handled by 2 bi-directional Long-Short Term Memory (LSTM) layers each has 128 units to extract temporal features.

Two outputs from the feature extractors are combined in an attention head. Here there are two approaches. The first one is Bahdanau's type attention, it is represented as below and this is the one I using for my attention block:

$$A_t = \text{Softmax}\big(\tanh\big(\text{Dot}(E_{\text{yam}}, h_t)\big)\big)$$

$$C_{\text{att}} = \sum A_t h_t$$

$$E_{\text{combined}} = \text{concat}(E_{\text{yam}}, C_{\text{att}})$$

$$y_{\text{pred}} = Sigmoid\big(E_{\text{combined}}^{\top} P + b\big)$$

E_{yam} is the output of the Yamnet feature extractor after the projection layer, h_t is the hidden state of the t^{th} sequence of the Wave Encoder output, and A_t is the attention vector. The attentive context vector is C_{att}. Finally, C_{att} is concatenated with E_{yam} to obtain the final joint embedding.

The second approach is transformer attention block mechanism:

$$Q = E_{\text{yam}}^{\top} W_q + b_q$$

$$K = h^{\top} W_k + b_k$$

$$V = \tanh(Q + K)\, W_v + b$$

$$A_t = \text{Softmax}(V)$$

Lastly, after having joint embedding vector, classification is made by sigmoid function and loss calculation through BCE.

4.2 Implementation

The paper's model classification is made by sigmoid function and loss calculation through BCE but in my implementation, I use BCEWithLogits because I want to calculate the other metrics like mAUC, dPrime,... so the output of my classifier head is logits.

The hyperparameters:

- $YAMNET\_OUT\_DIM = 1024$

  Explain: Output dimension of Dense layers

- $PROJ\_DIM = 256$

  Explain: Projection layers output's dimension

- $LSTM\_HID = 128$

  Explain: Unit inside each LSTM

- $LSTM\_DIRECTIONS = 2$

  Explain: Bidirectional LSTM

- $LSTM\_OUT\_DIM = LSTM\_HID * LSTM\_DIRECTIONS$

  Explain: Wave Encoder output's dimension (256)

- $NUM\_CLASSES = 1$

  Explain: Number classes of each prediction (single label classification)

The number of CNN layers in the Yamnet feature extractor is 10 layers, each has a convolutional Conv2d, a BatchNorm2d for normalization and a ReLU. In total, my LEAN model has 6,231,137 parameters which is slightly larger than the model in the paper with almost 5 millions parameters.

5 Model Evaluation

For metrics, AUC, mAP and dPrime are metrics used for evalutaion. AUC ROC is a curve that represents the ratio between true positive and false positive and if it closes to 1.0, that means predictions are valid and maybe the model has found the right way to find the label for each audio clip. mAP is mean average precision that calculates the mean of the average precision of each class and it is best for detection. dPrime is a sensitivity score that marks how many time the class is chosen and distinguish a signal from noise so it is really necessary for model trained with augmented data.

Training process with learning rate = 0.0005 and 25 epochs so it will cover all the possibility to improve accuracy. Moreover, I have early stopping mechanism with patience equals 3 so if after 3 epochs, accuracies are not improved, the training process stops. At this point, there are only 2 layers of CNN to extract the features so it might be the reason validation loss is so high and apparently overfitting.

Train No.	Total Epoch	Best Epoch	Train Acc	Train Loss	Val Loss	AUC	mAP	dPrime
1	5/25	2	0.9353	0.1995	0.2025	0.9915	0.9912	3.349
2	14/25	11	0.9908	0.0470	0.0274	0.9997	0.9997	4.804

Table 1: Comparation between 2 best training progress

As the result, we can see that after 5 epochs, the training process stop which means the most optimal epoch is epoch number 2. The train loss is quite large with 0.1995 and validation loss is also slightly large with 0.2025. Because the best epoch is to early so the model is unable to extract more features to improve classification. However, other metrics are better when Accuracy is 0.9353, mAUC = 0.9915, mAP is 0.9912 and dPrime is 3.349 (which is better than 2.251 in the paper with FSD50K dataset).

At the testing process, three main metrics are mAUC, mAP and dPrime quite the same with the training process about 0.9857, 0.9841 and 3.0788 respectively.

These numbers are when the model is trained with only 2 layers of CNN in feature extractor and high learning rate. Model performance improves when the learning rate is set to 1e-4, weight decay to 1e-2, and eight extra layers are added to YAMNet. Surprisingly, the training values look even better. Training loss and Validation loss have decreased dramatically and set at under 0.05 which is splendid. Accuracy is also better, approximate 0.99, AUC, mAP and dPrime are nearly perfect.

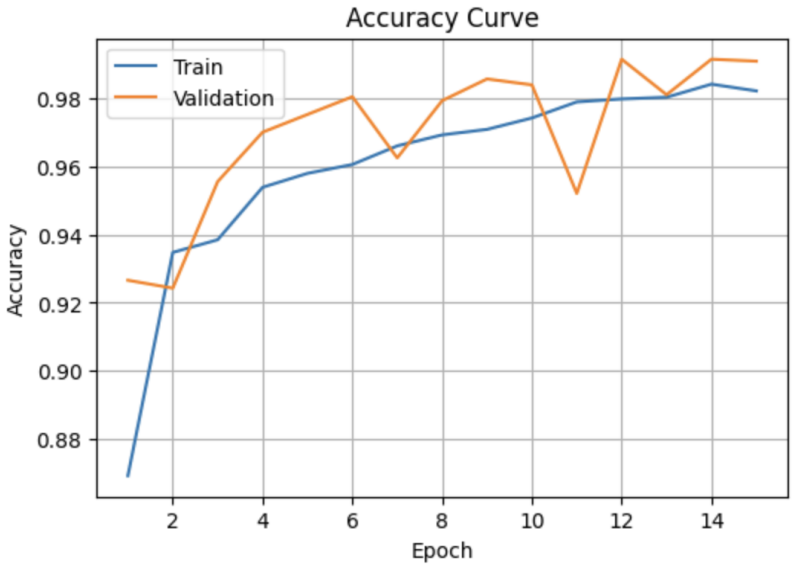

Figure 4: Accuracy on training and validation progress

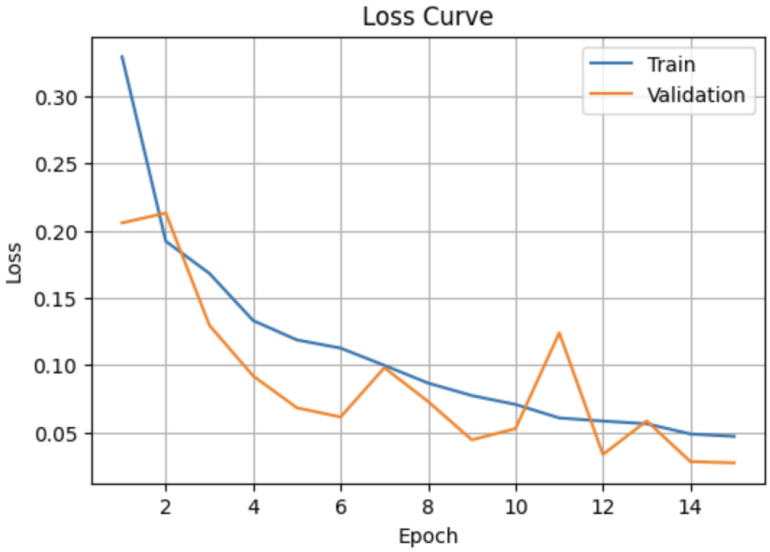

Figure 5: Loss on training and validation progress

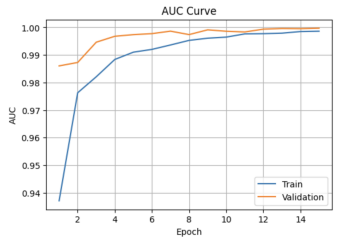

Figure 6: AUC on training process

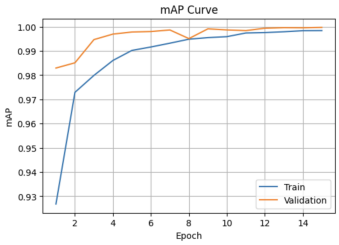

Figure 7: mAP on training process

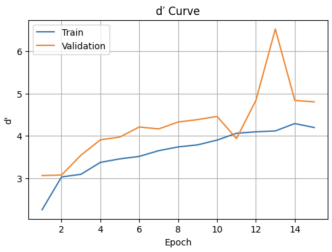

Figure 8: dPrime on training process

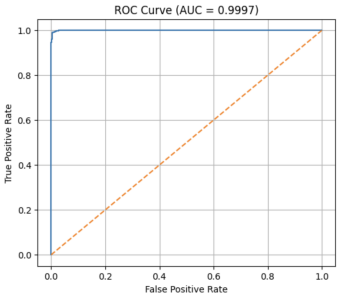

Figure 9: AUC on testing process

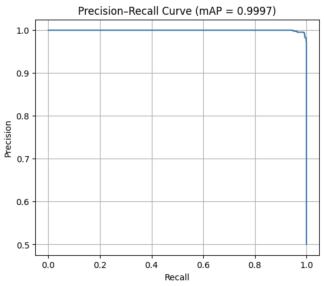

Figure 10: mAP on testing process

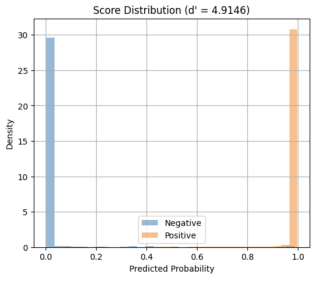

Figure 11: dPrime on testing process

6 Conclusion

6.1 Overall

Streamlit is a library that provides an easy low-code web implementation that has upload files and microphone input so the model can make prediction from both formats. The former is online audio file from various source but mainly are recorded in good condition which is low or no-noise environment and clear voice so the model predicts very well on file upload. On the other hand, input audio clip is harder to predict because the environment noise and interference are unpredictable. Even though the training dataset has been noised and augmented but the amount of samples is the limitation for model to learn better when the audio is too noisy.

6.2 Achievement

For more information, when upload a file for example an Indian male audio sample, the model predict it is male with 99.68% confidence and also 99.63% with an example female voice sample. The first one is on Kaggle dataset which is not the dataset the model is trained on and the second example is from the LibriVox which is an open source audio domain audiobook. The length of the audio file does not effect the accuracy of the prediction that has been tested with an 1 minute 37 seconds audio clip with Alessia Clara from Border Crossing and it still predicts it is female with 99.16%.

6.3 Limitation

The first limitation of this project is low samples on training dataset. In total of male and female training samples are 8075 which is not too optimal for training with augmentation dataset and the model is not able to calculate enough weight for predicting sample with plenty of noise. This might be the biggest reason that cause the misprediction of the microphone input function which contains too many noise from various sources like computer hardware, household, and other things. For example, if the sample has noise like an office with plenty of people talking but all female, the model still predict it is male with high confidence about 99.57%. However it is impossible for free GPU cloud notebook to handle bigger dataset because the model has trained in 18 thousands samples and get GPU crashed so downsampling is the only way to train and evaluate the model under budget. In future work, with better GPU resource and computer hardware, the model may develop and handle more type of data input with better noise suppression or extract better features so it has better weights to make prediction from noise data.

A Code and Web Demo

The source code for the backbone of LEAN model which is JointModel, data processing and training notebook is publicly in my Github repository.

- **Github Repo:** Integrated-Project—LEAN-Model-for-Audio-Gender-Classification

For demo, a web-based application was developed using Streamlit to provide an interactive demonstration of audio gender classification.

- **Streamlit Web Demo:** `https://genderclassify.streamlit.app/`

References

[1] AI Engineering Intern, Speech Recognition - 2026 https://www.linkedin.com/jobs/search/?currentJobId=4339379740&keywords=nvdia&origin=BLENDED_SEARCH_RESULT_NAVIGATION_JOB_CARD&originToLandingJobPostings=4339490024%2C4339440103%2C4339379740

[2] Audio Signal Classification: History and Current Techniques https://www2.cs.uregina.ca/~gerhard/publications/TRdbg-Audio.pdf

[3] prithivMLmods/Common-Voice-Gender-Detection https://huggingface.co/prithivMLmods/Common-Voice-Gender-Detection

[4] Audio Transformers https://arxiv.org/abs/2105.00335

[5] LEAN: Light and Efficient Audio Classification Network https://arxiv.org/pdf/2305.12712